Program:

```
def encode(d):
    """d: list of 4 bits [d1, d2, d3, d4]
    -> return 7bit hamming code list
    Positions (1-indexed): 1=P1 2=P2 3=d1 4=P4 5=d2 6=d3 7=d4"""
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c3 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
    if error_pos != 0:
        print(f"Error detected at bit position: {error_pos} -> correcting...")
        h[error_pos-1] ^= 1
    else:
        print("No error detected.")
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)
```

Expno: 6 Error Detection and
Date: 08/8/24 Correction using Hamming
code

Aim:
To write a program to implement error detection and correction using the Hamming code.

Algorithm
1. Start the program
2. Read the number of 4-bit data blocks (n) forming the input data stream.
3. For each 4-bit data block D = d1 d2 d3 d4, perform Hamming encoding.
a. arrange 7 bits with parity bits at positions 1, 2 and 4, and data bits at positions 3, 5, 6 & 7.
b. Compute the parity bits using even parity:
$P_1 = d1 \text{ XOR } d2 \text{ XOR } d4$
$P_2 = d1 \text{ XOR } d3 \text{ XOR } d4$
$P_3 = d2 \text{ XOR } d3 \text{ XOR } d4$
4. Simulate transmission by introducing a single-bit error at a chosen bit position.
5. At the receiver, recompute the check bits.
$C_1 = P_1 \text{ XOR } d1 \text{ XOR } d2 \text{ XOR } d4$
$C_2 = P_2 \text{ XOR } d1 \text{ XOR } d3 \text{ XOR } d4$
$C_3 = P_4 \text{ XOR } d2 \text{ XOR } d3 \text{ XOR } d4$
6. Compute Error Position = $C_4C_2C_1$ (binary number formed from the check bits)

```
def main():
    print("===HAMMING CODE - ERROR
    DETECTION AND CORRECTION ===\n")
    n = int(input("Enter a number of 4-bit
data blocks in the stream: "))

    for b in range(1, n+1):
        print(f"\n---block {b}---")
        d = list(map(int, input("Enter 4 data bit
        (space separated, eg 1011): ").split()))
        h = encode(d)
        print(f"original Data bits :
    {to_str(d)}")
        print(f"Encoded Hamming code (7 bits
        : {to_str(h)}")
        if 1<= pos <=7:
            h[pos-1] ^= 1
            print(f"Transmitted code (with error
induced at pos {pos}): {to_str(h)}")
        else:
            print(f"Transmitted code (no error detected)
        rd = [h[2], h[4], h[5], h[6]]
        print(f"Recovered data bits:
    {to_str(h)}")

if __name__ == "__main__":
    main()
```

6. Compute Error position = $C_4 C_2 C_1$ (binary number formed from the check bits).

7. If Error position = 0, no error has occured. Else, the bit at "Error position" is inverted (complemented) to correct the error

8. Extract the original data bits from the corrected 7-bit code

9. Repeat steps 3 to 8 for all n blocks in the data stream

10. Stop the program.

OUTPUT

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1011): 1011
Original Data Bits     : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code          : 0100011
Checking for errors...
  Error detected at bit position: 3 -> Correcting...
Corrected Code         : 0110011
Recovered Data Bits    : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1011): 0110
Original Data Bits     : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code          : 1100100
Checking for errors...
  Error detected at bit position: 6 -> Correcting...
Corrected Code         : 1100110
Recovered Data Bits    : 0110
```

Result:

The program to implement error detection and correction using the Hamming code concept was successfully written in python, executed & verified using a sample input data stream.